

Building and Securing a REST API

MoMo SMS Data Processing System

Team: Aimable (Lead), Richard, Jean de Dieu Tuyishime, Eloi

30 September 2026

Repository: <https://github.com/BANCUNGUYE66/momo-sms-processing-system>

1. Introduction to API Security

The MoMo SMS processing system reads mobile-money text messages from `data/raw/modified_sms_v2.xml`, turns each message into a transaction record, and now exposes those records through a REST API. A transaction record holds an id, a type, an amount in RWF, a sender, a receiver, a timestamp, and the original message text. That is financial data. A client that can list, create, change, or delete it should have to prove who it is first.

The API is a plain-Python server built with the standard-library `http.server` module (`api/rest_api.py`). It does not use a web framework. Every path under `/transactions` is protected with HTTP Basic Authentication, implemented in `api/auth.py`. The server reads the `Authorization` header, base64-decodes the `username:password` pair, and compares both values with the configured credentials using a constant-time comparison. The demo account is `admin / momo1234`. Those values can be replaced with the `API_USERNAME` and `API_PASSWORD` environment variables.

If the header is missing, malformed, or does not match, the server does not return transaction data. It responds with **401 Unauthorized**, a `WWW-Authenticate: Basic realm="MoMo SMS API"` header, and a JSON body: `{"error": "Unauthorized: invalid or missing credentials"}`. A request that authenticates successfully can then use the CRUD endpoints. Creating, updating, and deleting still check the body and the id, and return 400 or 404 when those are wrong. Authentication and input checking are separate steps.

Basic Authentication is enough to stop an anonymous client, which is what this assignment requires. It is a weak scheme for a real payment system. Section 4 explains why, and why JWT or OAuth2 would be a stronger design. The practical minimum, even with Basic Auth, is to send it only over HTTPS so the encoded password is not visible on the network.

2. Documentation of Endpoints

The server listens on `http://localhost:8001` when started with `python -m api.rest_api`. The store is seeded from the parsed XML. Records are kept in a dictionary keyed by id, so a lookup by id does not scan the full list. The same dictionary is what the DSA comparison in Section 3 measures.

GET /transactions

Lists transactions. Optional query parameters `limit` and `offset` page the list.

```
curl -u admin:momo1234 "http://localhost:8001/transactions?limit=2"
```

200 OK

```
[
  {
    "id": 1,
    "type": "RECEIVED",
    "amount": 2000.0,
    "sender": "Jane Smith",
    "receiver": "Own Account",
    "timestamp": "2024-05-10T16:30:58.724000",
    "raw_text": "You have received 2000 RWF from Jane Smith ..."
  }
]
```

Error: 401 if credentials are missing or wrong.

GET /transactions/{id}

Returns one transaction. The id in the path must be an integer.

```
curl -u admin:momo1234 http://localhost:8001/transactions/1
```

200 OK

```
{
  "id": 1,
  "type": "RECEIVED",
  "amount": 2000.0,
  "sender": "Jane Smith",
  "receiver": "Own Account",
  "timestamp": "2024-05-10T16:30:58.724000",
  "raw_text": "You have received 2000 RWF from Jane Smith ..."
}
```

Errors: 400 if the id is not an integer, 401 if credentials are missing or wrong, 404 if that id does not exist.

POST /transactions

Creates a transaction. `amount`, `sender`, and `receiver` are required. `type` defaults to `OTHER`, `timestamp` defaults to the current time, and `raw_text` is optional.

```
curl -u admin:momo1234 -X POST http://localhost:8001/transactions \
-H "Content-Type: application/json" \
-d '{"amount": 5000, "sender": "Alice", "receiver": "Bob", "type": "TRANSFER"}
```

201 Created

```
{
  "id": 1694,
  "type": "TRANSFER",
  "amount": 5000.0,
  "sender": "Alice",
  "receiver": "Bob",
  "timestamp": "2026-09-25T09:00:00.000000",
  "raw_text": ""
}
```

Errors: 400 if the JSON is invalid, a required field is missing, or `amount` is not numeric. 401 if credentials are missing or wrong.

PUT /transactions/{id}

Updates a transaction. The body may include any subset of `type`, `amount`, `sender`, `receiver`, `timestamp`, and `raw_text`. Only the fields that are sent are changed.

```
curl -u admin:momo1234 -X PUT http://localhost:8001/transactions/1694 \
-H "Content-Type: application/json" \
-d '{"amount": 7500}'
```

200 OK — the same record, with `amount` now `7500.0`.

Errors: 400 if the id is not an integer, the JSON is invalid, or `amount` is not numeric. 401 if credentials are missing or wrong. 404 if that id does not exist.

DELETE /transactions/{id}

Deletes a transaction. A successful delete has an empty body.

```
curl -u admin:momo1234 -X DELETE http://localhost:8001/transactions/1694
```

204 No Content

Errors: 400 if the id is not an integer, 401 if credentials are missing or wrong, 404 if that id does not exist.

Error codes

Code	When it is returned
200	Successful GET or PUT
201	POST created a transaction
204	DELETE removed a transaction; the body is empty
400	Bad id, invalid JSON, or a missing or non-numeric field
401	Missing or wrong Basic Auth credentials
404	No transaction with that id
405	That HTTP method is not allowed on the path

These codes were checked against a running server. An authorized `GET /transactions?limit=1` returned 200. The same path with username `wrong` and password `badpass` returned 401 and the `WWW-Authenticate` header. A POST created a record (201), a PUT changed its amount (200), and a DELETE removed it (204). A following GET of that id returned 404. The curl transcripts and screenshots are in `docs/evidence/` and `screenshots/`.

3. Results of the DSA Comparison

`dsa/xml_parser.py` reads every `<sms>` element in `modified_sms_v2.xml` and writes a list of dictionaries to `data/processed/transactions.json`. The run used for this comparison produced 1,691 records. Each dictionary has `id`, `type`, `amount`, `sender`, `receiver`, `timestamp`, and `raw_text`. Types are `RECEIVED`, `PAYMENT`, `TRANSFER`, `DEPOSIT`, `AIRTIME`, or `OTHER` when the message text does not match a known pattern.

The search question is: given an id, how do we find that one record? `dsa/search.py` implements two answers.

Linear search walks the list from the first record to the last and compares `id` on each step. The best case is one comparison, when the target is first. The average and worst cases grow with the length of the list, so the cost is $O(n)$. Extra memory is $O(1)$. A missing id still costs a full scan of all 1,691 records.

Dictionary lookup first builds a Python dictionary, `id → record`, in one $O(n)$ pass. After that, finding a record is a hash-table lookup. The average cost of a lookup is $O(1)$: the id is hashed and the bucket is read directly. The dictionary uses $O(n)$ extra memory. A missing id still takes one lookup, not a scan. This is the structure `api/data_store.py` uses for GET, PUT, and DELETE by id.

`dsa/benchmark.py` timed both methods on the full 1,691-record set. It chose 20 ids spread from the start of the list to the end, plus the missing id `999999`. Each id was searched 500 times and timed with `time.perf_counter_ns()`. The table reports the average time per search.

Target id	Found	Linear scans	Linear time	Dict scans	Dict time	Speedup
1	Yes	1	0.595 μ s	1	0.463 μ s	1.28×
90	Yes	90	10.383 μ s	1	0.221 μ s	46.94×
179	Yes	179	22.582 μ s	1	0.165 μ s	137.03×
268	Yes	268	33.580 μ s	1	0.163 μ s	206.52×
357	Yes	357	40.509 μ s	1	0.155 μ s	261.01×
446	Yes	446	58.705 μ s	1	0.150 μ s	390.32×
535	Yes	535	103.440 μ s	1	0.180 μ s	574.67×
624	Yes	624	100.806 μ s	1	0.161 μ s	626.12×
713	Yes	713	98.026 μ s	1	0.156 μ s	627.57×
802	Yes	802	171.868 μ s	1	1.353 μ s	127.06×
891	Yes	891	181.467 μ s	1	0.189 μ s	958.11×
980	Yes	980	161.287 μ s	1	0.531 μ s	303.86×
1069	Yes	1069	222.196 μ s	1	0.163 μ s	1,366.52×
1158	Yes	1158	220.751 μ s	1	0.152 μ s	1,450.40×
1247	Yes	1247	243.183 μ s	1	0.159 μ s	1,529.45×
1336	Yes	1336	362.077 μ s	1	0.161 μ s	2,254.53×
1425	Yes	1425	465.841 μ s	1	0.515 μ s	904.90×
1514	Yes	1514	598.133 μ s	1	0.156 μ s	3,834.19×
1603	Yes	1603	269.661 μ s	1	0.174 μ s	1,548.00×
999999	No	1691	358.687 μ s	1	0.384 μ s	934.57×

Across those searches, linear search averaged **186.19 μ s**. Dictionary lookup averaged **0.28 μ s**.

Dictionary lookup was **647.55 times faster** overall. The gap is small for id 1, because linear search stops on the first record. It grows as the target sits further down the list. For id 1514, linear search did 1,514 comparisons and took 598 μ s, while the dictionary still did one lookup in 0.156 μ s. For a missing id, linear search walked all 1,691 records. The dictionary answered in one lookup.

Dictionary lookup is faster because a hash table computes a bucket from the key and reads that bucket. The work does not depend on how many records are stored, as long as collisions stay rare. Linear search repeats a comparison once per record until it hits the id or reaches the end, so a later record always costs more. The tradeoff is memory: the dictionary keeps a second structure of size $O(n)$. For this API that cost is appropriate, because almost every single-record request is a lookup by id.

Other structures could help in different cases. A list kept sorted by id can be searched with binary search in $O(\log n)$, about 11 comparisons for 1,691 records, with no extra index, but inserting a record into the middle of the array is $O(n)$. A balanced tree (AVL or red-black) keeps search, insert, and delete at $O(\log n)$ and also supports range queries, which a hash table does not. A B-tree is the usual on-disk index when the records no longer fit in a convenient memory dictionary. A second dictionary keyed by sender or type would make those filters $O(1)$ as well. None of those replace the id index the API uses today. They answer different queries.

4. Reflection on Basic Auth Limitations

Basic Authentication sends the username and password on every request. The pair is only base64-encoded. Base64 is an encoding, not encryption. Anyone who can see the HTTP request — on a network without TLS, in a proxy log, or in a server access log — can decode `Authorization: Basic ...` and recover the password. The demo header `Authorization: Basic YWRtaW46bW9tbzEyMzQ=` decodes directly to `admin:momo1234`.

The same password is reused forever. There is no expiry, no scope, and no way to cut off one client without changing the shared password for every client. The server also cannot tell two legitimate callers apart if they both use `admin`. That is acceptable for a class demo with one account. It is a poor fit for an API that mobile apps and other developers would call, which is the setting described in the assignment.

Two stronger designs:

- **JWT.** The client logs in once. The server returns a signed token with an expiry and, if needed, a role or scope. Later requests send the token, not the password. The server checks the signature and the expiry. A token can lapse on its own, and the password is not repeated on every call.
- **OAuth2.** A client receives a short-lived access token, and usually a refresh token, with a limited scope. A third-party app can be allowed to read transactions without ever receiving the user's password. Access can be revoked for that app alone.

If Basic Auth is kept, it should be used only over HTTPS, so the encoded password is inside TLS. The credentials in this project are demo values committed for the assignment. They should not be reused on a public server.

Source notes for this report: endpoint behaviour is implemented in `api/rest_api.py` and `api/auth.py` and written up in `docs/api_docs.md`. The search implementations, the 1,691-record benchmark, and the timing table are in `dsa/search.py`, `dsa/benchmark.py`, and `docs/dsa_analysis.md`.